Where the Fix Lives: Code-Graph Structure and the Triage Gap for Local Agents

Haresh G. (Kaggle: Haresh276) · Code, data and both notebooks: github.com/Haresh276/where-the-fix-lives

Abstract

Gemma 4 31B fails most of the bugs it is given, and the ones it fails also eat most of its 12 hours. I wanted to know what predicts a failure, and how early the agent itself could tell. First I built static code graphs for all 500 SWE-bench Verified tasks at their base commits and joined them with per-task results for 38 models. Two things about the fix location predict failure even after patch size is controlled. One is an issue that names nothing connected to the fix (−0.91 [−1.46, −0.40] log-odds). The other is a fix that spans more files (−0.34 [−0.65, −0.10] per SD). Neither shows up in the issue text (AUC 0.45–0.54 before search), and even knowing exactly where the true fix sits adds little. What separates wins from losses is how big the edit is (AUC 0.63–0.72). Then I ran Gemma 4 31B twice on all 129 public competition tasks, on the competition's own 4×L4 setup, and went through all 258 traces. Gemma opens a file the fix changes in 92% of task-runs, after a median of 12 seconds, and that predicts nothing (AUC 0.44). The thing that predicts success is when it first edits source code. That gets AUC 0.81, better than the true patch size (0.71), and it holds when one run's timing is used to predict the other run. Task-runs with no source edit by 300 of 360 seconds solved 1 of 66 and used 37% of all runtime. So finding the fix is easy for Gemma. Starting to write it is where things go wrong.

1. Introduction

The competition gives an agent 12 hours on four L4 GPUs and a hidden pile of bugs. In my first full run of Gemma 4 31B on the public tasks, failed tasks took twice as long as solved ones (median 364 s against 180 s) and used 80% of the runtime. So most of the budget goes to bugs the agent was never going to fix. If it could spot those early, it could spend the time on bugs it can actually fix.

Patch size is the obvious guess, and it does matter. But two three-line patches can be completely different jobs. In one, the issue names the broken function. In the other, the broken line is three calls away from anything the issue mentions, in a file nobody named. A code graph can measure that difference, and every agent in this competition already gets graph tools. So I asked three questions.

1. Does **where the fix sits in the code graph** predict failure, beyond patch size and issue length, across many models?
2. Can an agent see that **before** it searches, or only after it has found and planned the fix? I call the difference the *triage gap*.
3. For **Gemma 4 31B on the competition's own tasks**, what can the agent itself notice early enough to act on?

Related work. Agent Psychometrics [2] predicts task difficulty from features an LLM judges from the issue, patch and tests. Balepur et al. [3] show on synthetic call-graph tasks that understanding code limits coding agents more than editing it does. Agentless [7] and LocAgent [8] treat localization as its own stage. SWE-bench [4] and SWE-agent [5] supply the tasks and scaffold. I measure real repository structure instead of asking a model to judge it, compare it with [2]'s released features, and then look inside the agent's own trajectory.

2. Data and method

Tasks and outcomes. SWE-bench Verified [4] has 500 human-checked bug fixes from 12 Python repositories (Django alone is 46%). The leaderboard publishes per-task results for 40 runs of the bash-only *mini-SWE-agent* scaffold [5]. Two runs have per-task files that disagree with their own reported score by more than 5 points, so I dropped them. That leaves **38 models** (15 open-weight), resolving 9% to 77%. The runs span 11 scaffold releases, so every run gets its own fixed effect, and Section 3.1 also refits within single releases. *Weak* means the five models below 45% (Qwen2.5-Coder 32B, Llama 4 Maverick, gpt-oss-120b, GPT-5 nano, Kimi K2). *Strong* means the 13 at 70% or above.

Code graphs at every commit. For each task I parse every non-test Python file at its base commit, straight from a bare git clone and cached by file hash, into functions, methods and classes. These are linked by call edges and by class-to-method containment. Calls resolve by name, which over-approximates, like most static Python call graphs. The median graph has 9,308 nodes and the biggest has 31,488.

Where the fix lives. I map each task's curated fix spans onto graph nodes (97.5% resolve) and measure the following.

- *Hops* is the graph distance from anything the issue names (identifiers, traceback frames) to the fixed code. 0 means the issue names the fixed function. *No anchor* (72 tasks) means nothing the issue names reaches the fix within 8 hops.
- *Files* and *functions* the fix touches, and whether the issue names the fix *file*.
- Patch lines and issue length, as controls.

Example. In `sphinx-doc__sphinx-9591` the fix is three lines in one method, three hops from the nearest thing the issue names. All 13 strong models fix it. None of the five weak models do. Patch size alone would call it easy.

Analyses. A pooled logistic model of 19,000 task × model outcomes with a fixed effect per run, with 95% CIs from 1,000 task-level bootstrap resamples and leave-one-repository-out refits. The same model fitted per model. Ridge regression of [2]'s item-response difficulty. And classifiers of each weak model's solves at rising information levels, with whole repositories held out.

My Gemma runs. The public competition tasks are merged pull requests in fastapi, requests, rich and httpx. I ran Gemma 4 31B (QAT, the default agent and tools, vLLM on 4×L4) on all 129 in one Kaggle session, with 6 minutes, 40 tool calls and 70 turns per task, 10.8 GPU-hours in total. Then I ran it again with one change, an edit-plan checkpoint in the prompt (Section 4). I rebuilt every task from public git history and took its fix spans from the diff. On 150 SWE-bench tasks this method matches the curated fix files every time (91% precision and 83% recall at function level). 27 fastapi tasks import packages the sandbox does not have [6]. Gemma solved 0 of them, so I drop them and keep 102. Every run also saves a full trace per task, with each model turn and tool call timestamped. Section 3.5 uses all 258 of them.

3. Results

3.1 Where the fix sits predicts failure

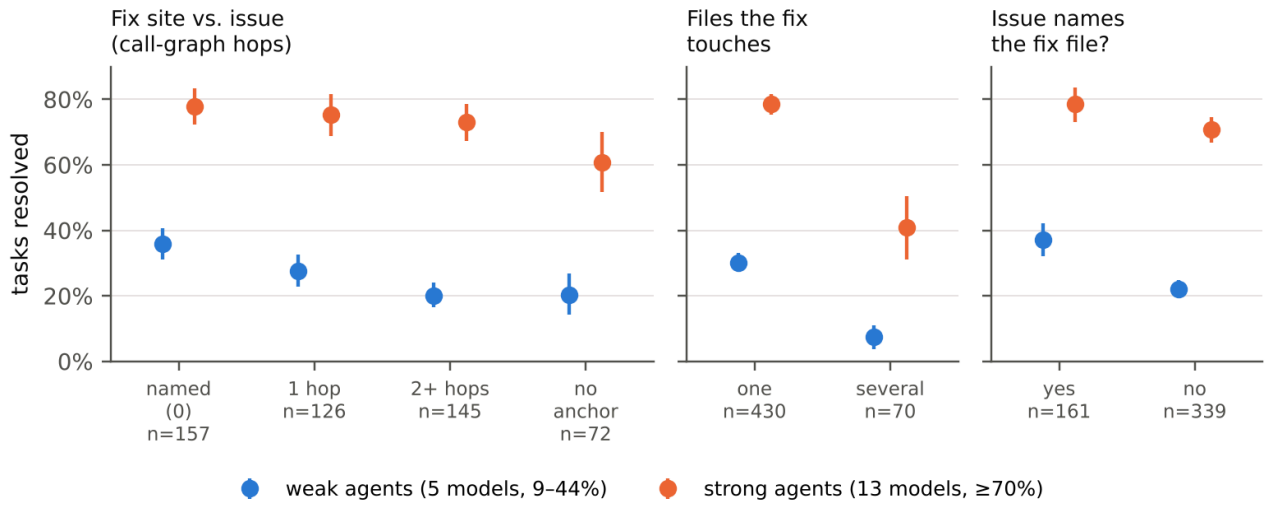


Figure 1. Resolve rate by where the fix lives, weak vs strong models (mean over models; 95% CIs from resampling tasks). SWE-bench Verified, 500 tasks.

With patch size, issue length and everything else held fixed, the strongest signal is a missing anchor. When nothing in the issue reaches the fix, the log-odds of success drop by −0.91 [−1.46, −0.40] (odds ×0.40). Among anchored tasks, distance points the same way, but I cannot separate it from zero (−0.15 [−0.36, 0.04] per SD). Using only call edges, without containment, changes nothing (−0.27 [−0.43, −0.11]).

Files touched has its own penalty (−0.34 [−0.65, −0.10] per SD, on top of patch lines). An issue that names the fix file helps a little (0.15 [−0.01, 0.33]). How spread out the fix is adds nothing once files and size are known (−0.01 [−0.22, 0.17]).

I tried hard to break these. Hops keep their sign with any one repository removed (12/12) and without Django (−0.28). Both effects also hold within single scaffold releases. v2.0.0 (11 models) gives hops −0.31 [−0.56, −0.08] and files −0.34 [−0.73, −0.06]. v1.7.0, home to three of the five weak models, gives −0.21 [−0.37, −0.08] and −0.44 [−0.79, −0.15].

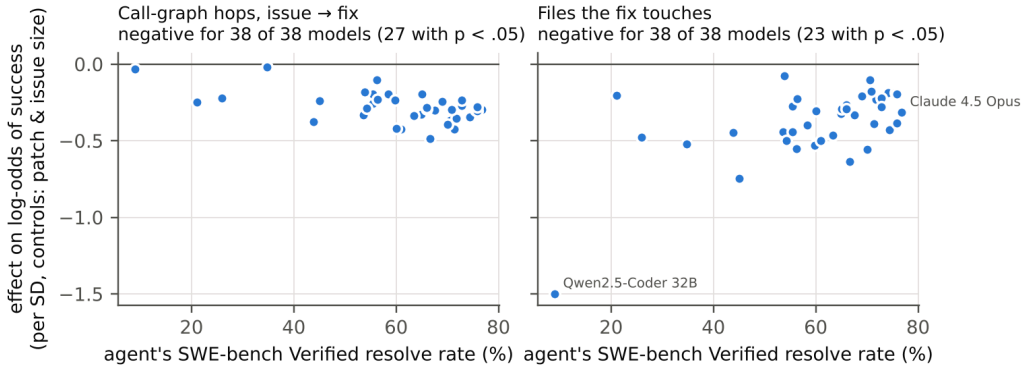


Figure 2. Per-model effect of hops (left) and files touched (right) on log-odds of success. Each dot is one of 38 models.

Per model, hops is negative for 38 of 38 and significant for 27 (Figure 2). The fits share the same 500 tasks, so this is a consistency check, not 38 replications. In raw rates, weak models resolve 30% of single-file fixes and only **7.4%** of multi-file fixes.

Do weak models suffer more? Not on the log-odds scale. Multi-file fixes cost weak models 1.68 log-odds and strong models 1.67. That is the same penalty. Weak models just start lower.

3.2 Static structure vs. an LLM judge

I lost this one, and I am reporting it anyway. For general task difficulty, [2]'s LLM-judged features reach Spearman ρ = 0.69. Patch size alone gets 0.40, my graph features plus patch size get 0.49, and adding mine to theirs gives 0.70. So the graph does not beat an LLM at "how hard is this task". It answers a narrower question, which is which known fixes the weak models miss.

3.3 The triage gap

Could an agent skip its hopeless tasks early? For each weak model I predict which tasks it solves, holding out whole repositories, at five levels of information.

1. The issue only.
2. Plus a CPU BM25 localizer's top hit and the graph around it.
3. Plus where the true fix sits.
4. Plus how many files and functions it spans.
5. Plus the true patch size.

Levels 3 to 5 use the true fix, so they are oracles. Level 4 is roughly what an agent knows once it has planned a correct edit.

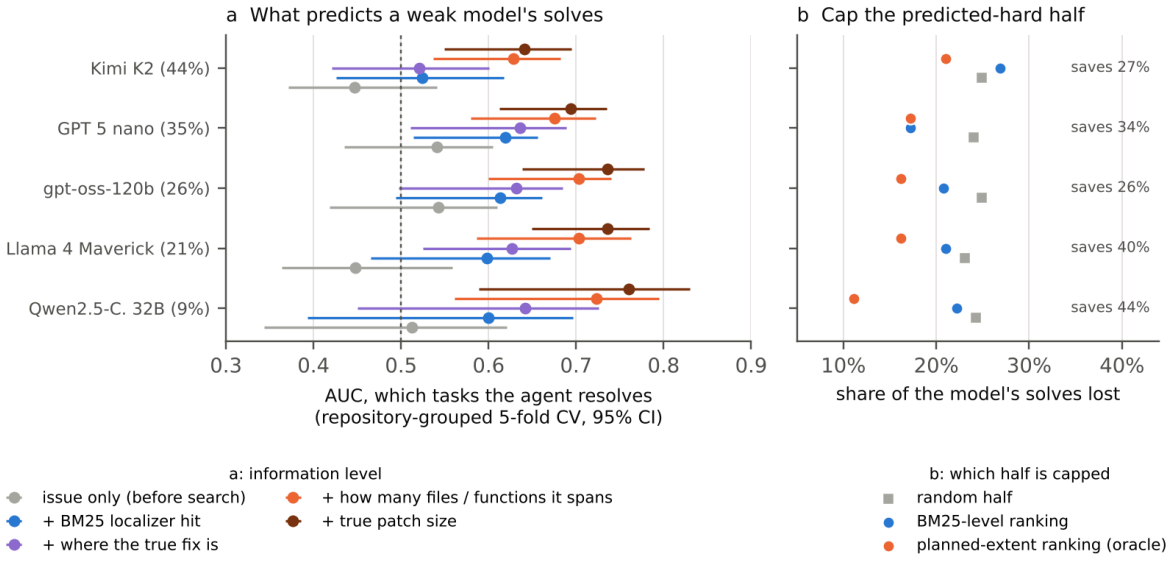


Figure 3. (a) AUC for predicting which tasks each weak model solves, by information level (repository-grouped CV; 95% CI from refitting on task resamples). (b) Solves lost when the predicted bottom half is capped at the model's median calls per solved task.

Before search, prediction is at chance or worse (AUC 0.45–0.54). The BM25 localizer finds the fix file first 44% of the time and lifts this to 0.52–0.62. Knowing exactly where the true fix sits barely helps (0.52–0.64), which honestly surprised me, since the whole paper started from the idea that location matters. The jump comes with the fix's extent, to 0.63–0.72, and 0.64–0.76 with the true patch size. **How big the edit is matters much more than where it is.**

I fixed one rule before looking at the results. Cap the predicted bottom half at the model's median calls per solved task, and count a solve as lost if it needed more. Ranked by extent, the cap saves 26–44% of calls and loses 11–21% of solves. A random half loses 23–25%. Weak

models spend 64–97% of their calls on tasks they fail, so this is not a small amount of wasted compute.

3.4 Gemma 4 31B on the competition's tasks

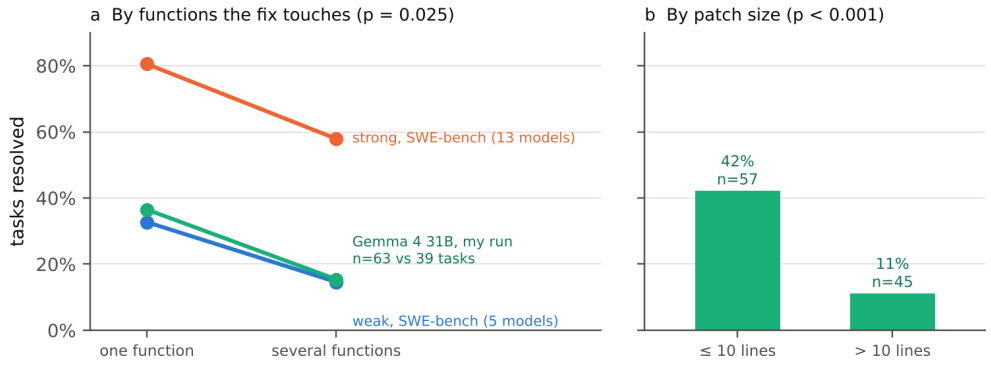


Figure 4. My first Gemma 4 31B run, 102 solvable competition tasks. (a) By functions the fix touches, with the SWE-bench cohorts. (b) By patch size.

Gemma solves 29 of 102 (28%). One-function fixes succeed **36%** of the time (n = 63) and multi-function fixes **15%** (n = 39, Fisher p = 0.025), the same drop the weak SWE-bench models show. Patch size splits harder, with 42% at 10 lines or fewer and 11% above (p < 0.001). Fitted together, patch size takes all of it (function count p = 0.979). Another entrant's CPU runs give the same split on their 55 tasks (50% vs 17%) [1]. 56 of 129 tasks hit the 6-minute wall.

3.5 Inside the runs: what the agent can see

Everything so far uses the true fix, which no agent has during a run. The traces show what Gemma actually did with its time.

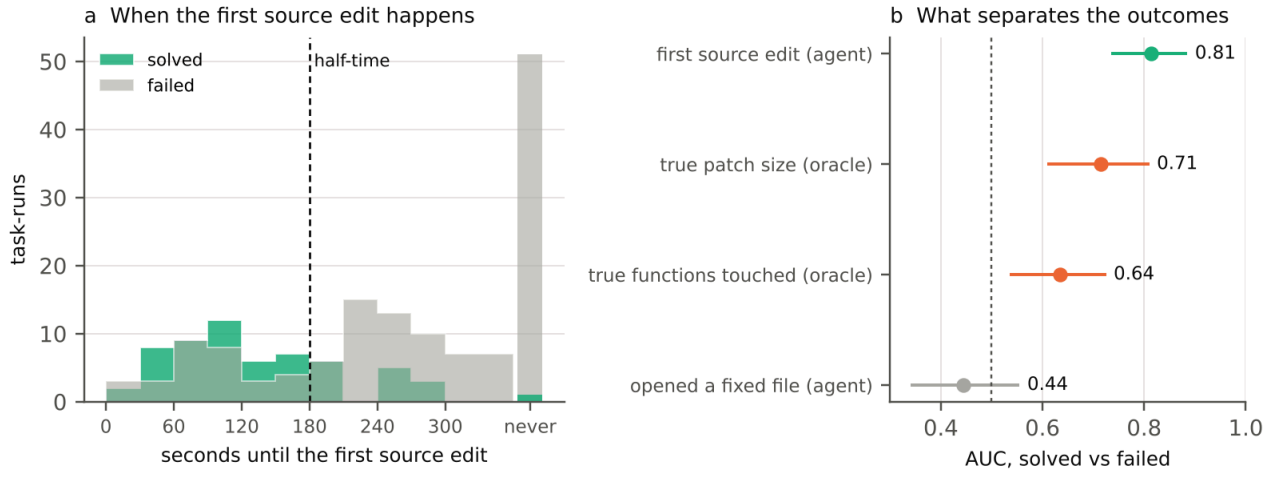


Figure 5. Both Gemma 4 31B runs, 99 tasks each. (a) When each task-run first edited a source file. (b) AUC for solved vs failed task-runs (95% CI from resampling tasks).

Finding the file is the easy part. Gemma opens a file the fix changes in 92% of task-runs, after a median of 12 seconds, usually with its very first tool call. How fast it gets there says nothing about whether it will succeed (AUC 0.44). This is Section 3.3 again, seen from inside the agent.

The time goes between finding and editing. The median first source edit arrives at 178 s, which is 148 s after the file was found. 77% of all runtime passes before any source edit at

all. The prompt goes past 14k tokens in 77% of task-runs, with only 16k available for input. Of 198 task-runs, 183 found the file, 134 edited it and 59 solved the task. Gemma gets to the right file in seconds and then spends most of the clock reading around it. I know this feeling from exam prep.

The first edit is the signal. The time of the first source edit separates solved from failed task-runs with AUC 0.81 [0.73, 0.89], better than the true patch size (0.71 [0.61, 0.81]). The agent can see this during a run, and the oracle still does not beat it. I wanted to be sure it was not just one run's clock fooling me. The two runs agree on 95% of outcomes, and first-edit time in one run predicts the *other* run's outcomes with AUC 0.76 and 0.81. Task-runs that edit before half-time (180 s) solve 60%, and the rest solve 12%. With no edit by 300 s, just 1 of 66 task-runs got solved, and those 66 used 37% of all runtime.

Its plans are honest but not very useful. In the checkpoint run Gemma wrote a plan in 67 of 99 tasks, and the plans name the right number of files 81% of the time. But plan size barely predicts the outcome (AUC 0.62, against 0.70 for true patch size on the same tasks). 14 of 23 fixes over 10 lines were planned as 10 lines or fewer. Gemma, like me, is an optimist about line counts. What does matter is whether a plan shows up at all. Tasks with no plan solved 1 of 32, even though 97% of them had found the file.

A stop rule. Stopping every task with no source edit at 180 s would save 29–30% of runtime and lose 21–29% of solves. Stopping as many by true patch size loses 29–39%, and at random 32–36%. So the agent's own clock does as well as the oracle or better. One more thing from the traces. The competition's code-graph tools were almost never used, 70 of 6,361 tool calls in 17 task-runs, and none of those task-runs were solved. Slightly awkward for a paper about code graphs.

4. What I would do with 12 hours

1. **Don't pre-screen from the issue.** Before search there is nothing useful to read.
2. **Don't reward finding the file.** Gemma finds it in seconds either way.
3. **Watch the first edit.** No source edit by half-time puts a task at 12%. Make the smallest edit at the best site, test once, and submit. No edit by 300 s means stop.
4. **Ask for a plan before the edit.** A plan that never shows up is the earliest warning sign. The plan's size is not a useful one.
5. **Give the saved time to tasks that are already editing,** by raising `max_time_minutes` for them.

I tested point 4 on the real setup. A second full run, identical except for the plan text in the prompt, solved 32 of 102 against 29 (4 gained, 1 lost, exact McNemar $p = 0.375$) and hit the 6-minute wall 46 times instead of 56. Right direction, not significant. One run per arm cannot tell a 3-task gain from luck, and I would rather say that than round it up into a win. Points 3 and 5 I only simulated. Running them costs one more Kaggle session. This fits a harness study which found that call quality matters more than call volume for Gemma 4 [9]. The first edit tells you which tasks to stop spending calls on.

5. Limitations

The SWE-bench outcomes come from one scaffold family across 11 releases. Within-release refits agree, but settings and dates differ, and SWE-bench tasks may be memorized. Figure 3's levels 3 to 5 use the true fix. Section 3.5 measures the agent's own view, but only for Gemma, only under a 6-minute cap, and it decides which tool calls are source edits using filename rules. Name-based call resolution over-approximates edges. My Gemma data are two runs of 102 tasks from four repositories. Another entrant reports 34.5% resolved locally

against about 6.9% on the hidden board [1], so local rates do not carry over to private repositories.

6. Release

Apache-2.0 at github.com/Haresh276/where-the-fix-lives. It has the commit-exact graph builder, the diff-to-span mapper, per-task features for all 500 SWE-bench Verified tasks and all 129 competition tasks, per-task outcomes and trace features for both Gemma runs, both notebooks, and one script (`paper_numbers.py`) that regenerates every number here on CPU. Every number in this paper is filled in from that script's output, not typed by hand.

AI-use statement: an AI assistant (Claude) wrote the code, ran the analysis, ran the Kaggle experiments on my account, and drafted this text in my style. I set the goal, directed the work, pushed back on the findings, and approved the final version.

References

- [1] yosh95. *g4-developer-agent-paper*: per-task Gemma 4 31B outcomes on public competition tasks. GitHub, 2026. Apache-2.0.
- [2] C. Ge, D. Kryvosheieva, D. Fried, U. Girit, K. Hariharan. Agent Psychometrics: Task-Level Performance Prediction in Agentic Coding Benchmarks. arXiv:2604.00594, 2026.
- [3] N. Balepur, K. Tomlinson, T. Schnabel. Code Understanding is a Bottleneck for Coding Agents. arXiv:2610.10610, 2026.
- [4] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024; OpenAI, SWE-bench Verified, 2024.
- [5] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024; mini-SWE-agent, 2025.
- [6] Kaggle competition forum, Gemma 4 Developer Agent Paper Track: "27 fastapi tasks cannot pass in the provided sandbox", Nov 2026.
- [7] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.
- [8] Z. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL 2025.
- [9] veljjanoski. *Better calls, not more calls: what limits a small coding agent on a fixed budget.* Gemma 4 Developer Agent Paper Track, Nov 2026; github.com/veljjanoski/gemma4-developer-agent.